COE MSDAE DADS5250 Scripts - Table of Contents

COE MSDAE DADS5250 Scripts - Table of Contents 1
- COE_MSDAE_DADS5250_M01V01_Why GenAI Matters 2
- COE_MSDAE_DADS5250_M03V01_Intro into Prompt Engineering 4
- COE_MSDAE_DADS5250_M05V01_LangChain Concept 5
- COE_MSDAE_DADS5250_M06V01_AI Agents 7
- COE_MSDAE_DADS5250_M08V01_RAG 9
- COE_MSDAE_DADS5250_M10V01_n8n 9
- COE_MSDAE_DADS5250_M11V01_Agentic AI + LangGraph 10
- COE_MSDAE_DADS5250_M13V01_AI Ethics 10

COE MSDAE DADS 5250 Scripts

Points to keep in mind:

- Compelling opening statement or hook (Why is this important to your organization and meaningful to you?)
- Goal of Video conveyed
- Succinct information - breaks and transitions between sections
    - Graphics & visuals discussed
- Relevance to industry
- Example or compelling story to demonstrate
- Evergreen content
- Summarize key points

500 words max = 5 minutes

COE_MSDAE_DADS5250_M01V01_Why GenAI Matters

PART 1: OPENING HOOK(Professor Intro)

Why does Generative AI matter for the future?

I want to start with a quick story. I was sitting at a café in Boston when the person next to me asked: "Have you used ChatGPT yet?" This was January 2023, just weeks after it launched. He insisted I log in right there. Within minutes, I was completely mind-blown… I just sat there, staring at the screen. I typed a follow-up. Then another. Each response came back sharper, more nuanced than the last. The coffee went cold. I didn't notice.

A machine that could read my questions, understand intent, and respond with the quality of a thoughtful colleague. I left that café calling friends, telling them to try it immediately.

That moment is the key moment that demonstrates why Generative AI…

PART 2: WHAT IS GENERATIVE AI? (ANIMATION)

Generative AI is a type of artificial intelligence trained on vast amounts of text and data that can produce original content in response to your instructions.

It sits within Deep Learning, which sits within Machine Learning, which sits within AI. The breakthrough that made it possible was a 2017 Google research paper called "Attention Is All You Need," which introduced the Transformer architecture. By 2022, OpenAI had used this foundation to release ChatGPT reaching one million users in five days. For context, Instagram took two and a half months to hit the same number.

"So what is Generative AI — in plain English?

Imagine you had a friend who had read every book, every article, every website on the internet. And whenever you had a question — any question — you could just ask them. And they would give you a thoughtful, clear answer. Right away. In plain language.

That is essentially what Generative AI is. You give it a question, or a task, and it responds — not by searching the web, but by actually thinking it through and generating something new for you.

Now this didn't just appear overnight. It took decades of research, building on top of itself, layer by layer. But the moment it all came together for the general public was late 2022, when ChatGPT was released. It reached one million users in just five days. For context — Instagram, one of the most viral apps ever built, took two and a half months to reach that same number.

That's how hungry people were for this."

PART 3: TWO REAL-WORLD EXAMPLES(ANIMATION)

Let me give you two examples of what this technology is actually doing right now not in a research lab, but in the real world.

Example one: drug discovery. Traditionally, developing a new drug from scratch takes an average of four to five years just to reach human trials and costs hundreds of millions of dollars. A company called Insilico Medicine used a generative AI platform to design a drug for a rare and deadly lung disease called idiopathic pulmonary fibrosis. They went from identifying the target to a drug candidate ready for human clinical trials in just 18 months, at a fraction of the typical cost. That drug is now in Phase II trials. The same AI approach is being applied to cancer, Alzheimer's, and dozens of other diseases. Treatments that might have taken a decade to discover are now being designed in months.

Example two: software development. In 2025, Google's CEO revealed that more than 25% of all new code written at Google is now generated by AI. At Microsoft, GitHub Copilot, an AI coding assistant, is used by over one million developers every day. These developers are not being replaced; they are completing tasks two to three times faster, spending less time on repetitive syntax and more time on creative problem-solving. The job of a software engineer is changing in real time.

A life-saving drug discovered in 18 months. A quarter of Google's code is written by AI. This is not a future scenario — it is March 2026, and this is already happening.

Let me show you what I mean — because the way we do everyday things has already changed.

A few years ago, if you needed to build a website, you hired a developer. If you needed a presentation, you spent hours on PowerPoint. If you wanted to build an app, you needed to know how to code. Today? You just describe what you want — in plain English — and AI does it for you.

Take Gamma. You type a topic — say, 'climate change for a business audience' — and in seconds you have a fully designed, professional presentation. No templates, no formatting, no design skills needed.

Or take vibe coding. Students right now — with zero coding experience — are building real, working apps just by describing what they want to an AI. You type 'build me a to-do app with a clean design' and it writes every single line of code for you. You just give feedback and direct it like a conversation.

Want a personal portfolio website? There are tools where you answer a few questions and your website is live in minutes. Want to start a business? AI can write your pitch deck, your business plan, your emails, and your social media content — all in an afternoon.

These are not futuristic scenarios. Your classmates are doing this right now. And the students who learn how to use these tools well — who understand what is happening under the hood — are the ones who will move fastest."

PART 4: WHY THIS MATTERS FOR YOUR CAREER (ANIMATION)

Here is the conclusion I keep hearing from industry leaders and hiring managers:

AI may replace some jobs but it will create opportunities for the people who know how to use it.

A colleague in life sciences put it well: "AI is going to be like Excel." In the early 2000s, spreadsheet literacy became a baseline expectation in every industry. AI literacy is heading in the same direction fast.

In 2024, the Nobel Prize in Physics went to the founders of neural networks, the very architecture powering the tools you will use in this course. That recognition tells you how significant this moment is.

"So here's the question I know some of you are sitting with — if AI can write code, build apps, design presentations, and discover drugs… what's left for engineers?

Everything that actually matters.

AI is an incredibly powerful tool. But a tool still needs someone who knows what problem to solve, why it matters, and whether the output is actually correct. That person is you. An engineer who understands AI doesn't get replaced — they become ten times more valuable.

Think about it this way. A calculator didn't replace mathematicians. It freed them to solve harder problems. AI is doing the same thing for engineers right now.

And that is exactly what this course is about. Not using AI as a novelty — but using it as an engineer. Knowing when to use it, how to direct it, and how to evaluate whether what it gives you is trustworthy.

Over the next few months, we are going to cover how AI can help you analyze data faster than any spreadsheet ever could. How it can help you build intelligent tools and pipelines. How it can even help you tackle complex optimization problems — like routing, scheduling, resource allocation — problems that engineers spend entire careers on.

The goal is simple. By the end of this course, AI is not something that intimidates you or replaces you. It is something you know how to wield — as an engineer, as an analyst, as a problem solver.

That skill, right now, in 2026, is one of the most valuable things you can walk out of a classroom with."

PART 5: WHAT YOU WILL BUILD

By the end of this course, you will be able to:

- Connect to AI models programmatically using the OpenAI and Gemini APIs.

- Design prompts that produce reliable, structured outputs.
- Build Retrieval-Augmented Generation pipelines grounded in your own documents.
- Create agent-based systems that reason and take actions autonomously.
- Automate real workflows by integrating AI into tools like n8n.

These are working applications - the kind you can put in a portfolio or deploy for a real use case.

COE_MSDAE_DADS5250_M03V01_Intro into Prompt Engineering

PART 1: Introduction: (Prof talking)

Now before we dive in, there is one important thing to understand about how these models actually work — because it changes everything about how you interact with them.

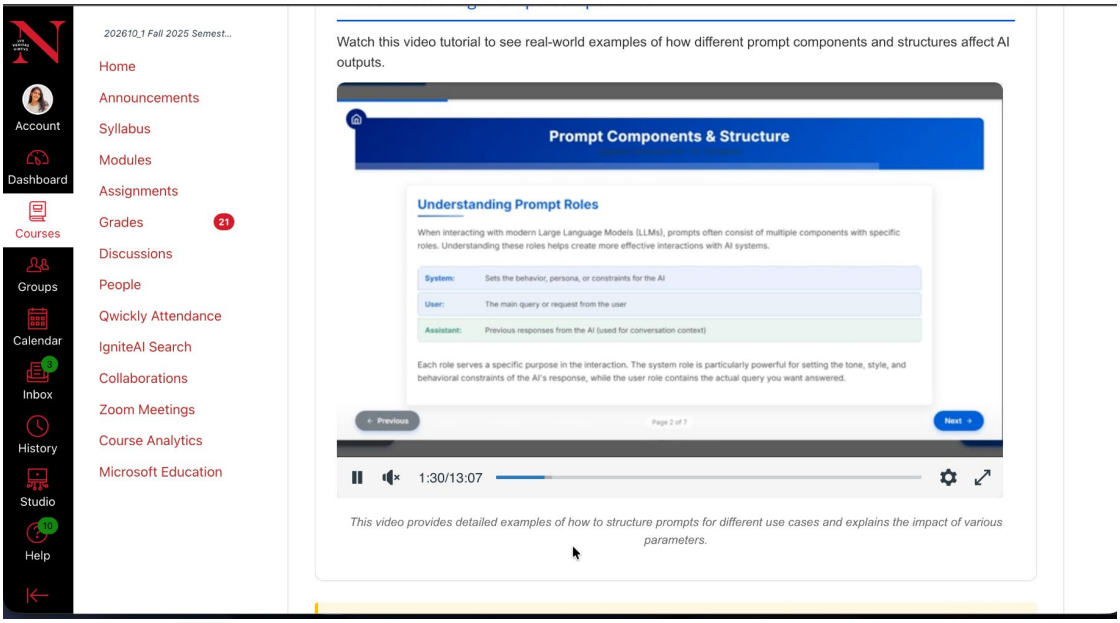
Generative AI is what we call a probabilistic model. That simply means it does not follow a fixed script. Every time you give it an input, it is calculating the most likely response based on everything it has learned. It is not looking up an answer — it is generating one. And here is the key part — the quality of that output depends almost entirely on the quality of the instructions you give it. Give it vague instructions, you get a vague answer. Give it clear, well-structured instructions, and the output becomes something genuinely useful.

That is exactly why the concept of prompt engineering exists.

Every morning I place the same coffee order: medium dark roast, whole milk on the side, cinnamon on top. My barista knows it by heart. I never have to explain myself twice. That is exactly the standard we should hold ourselves to when we write prompts for an AI model.

ANIMATION STARTS(Prof Voiceover): Simple flow diagram — User → Prompt → LLM → Output

PART 2: Prompt Structure: The Three-Layer Framework:



Before we can improve our prompts, we need to understand their structure. Every prompt sent to a large language model API has three components.

- The first is the **System** layer. This defines the model's role, persona, or behavioral constraints. For example: "You are a data analyst who provides concise, evidence-based responses and avoids speculation."
- The second is the **User** layer. This is the actual request. It tells the model what you need it to do. For example: "Summarize the three most significant findings in this quarterly earnings report."
- The third is the **Assistant** layer. This captures the model's prior responses. In multi-turn conversations, it can also be pre-filled to guide how the model continues.

Together, these three components form the anatomy of every API call you will build in this course.

PART 3: Essential Elements of a Well-Designed Prompt:

Beyond the three-layer structure, a well-crafted prompt should consistently include four essential elements.

- Role — Define who the model is acting as. This sets the behavioral frame for the entire response.
- Task — Specify exactly what needs to be done. Ambiguity here produces ambiguous outputs.
- Context — Provide relevant background, constraints, or data. The model cannot assume information that is not in the prompt.
- And Format — Indicate the expected output structure: a numbered list, a JSON object, a paragraph, or a table. If you do not specify a format, the model will choose one for you, and it may not be what you need.

The difference in output quality between these two prompts is significant. And it is entirely within your control as the person writing the prompt.

Part 4: Prompting Strategies: Zero-Shot, Few-Shot, Chain of Thought

Now let us look at three foundational strategies that determine how you guide the model toward the response you need.

Zero-shot prompting means asking the model to complete a task with no examples provided. You rely entirely on the model's existing knowledge. This is appropriate for straightforward tasks where the model has strong prior training, such as translation, simple summarization, or basic classification.

Few-shot prompting means including one or more labeled examples before your actual question. This guides the model toward the reasoning style, tone, or format you expect. It is particularly valuable when consistency and accuracy matter across many inputs.

Chain-of-thought prompting instructs the model to reason through the problem step by step before arriving at a final answer. Research published by teams at Google and OpenAI has

demonstrated that this approach significantly improves performance on complex tasks, including multi-step mathematical reasoning, logical inference, and structured decision-making.

Part 5: Why This Matters in Practice

These strategies are highly applicable in the real world In 2025 and 2026, organizations across industries are embedding structured prompt engineering directly into their AI-powered workflows. Well-designed prompts are the difference between a system that performs reliably at scale and one that produces inconsistent results requiring constant human correction.

In the lab associated with this module, you will apply all three strategies across different task types, including sentiment analysis, mathematical reasoning, and content generation, and you will compare the outputs directly.

COE_MSDAE_DADS5250_M05V01_LangChain Concept

PART 1: INTRODUCTION

I was at a symphony concert last year. Dozens of musicians on stage, each playing a completely different instrument. Individually, each one is talented. But without the conductor standing at the front, coordinating every section, every tempo change, every entrance, it would just be noise. LangChain is that conductor. It takes your language models, your memory, your tools, and your prompts, and it makes them work as one.

PART 2: So, What Is LangChain? When you send a prompt directly to a language model through the API, you are doing one thing: asking a question and getting an answer. That works for simple tasks. But real-world AI applications are rarely that simple.

You might need the model to remember what the user said three messages ago. You might need it to search an external database before responding. You might need it to call a tool, format the output, and pass it to a second model. Doing all of that manually, from scratch, for every application you build, is slow, repetitive, and error-prone.

LangChain solves this. It is an open-source framework that gives you pre-built components for every part of an AI pipeline, and a standard way to connect them together. Instead of writing all of that infrastructure yourself, you assemble it from well-tested building blocks.

PART 3: The Four Core Elements

LangChain is built around four core elements. Understanding each one is essential before you start building.

- **First, LLMs:** LangChain connects to multiple language models through a unified interface. You can use GPT, Gemini, or open-source models from Hugging Face, all with the same code structure. Switching models requires changing one line, not rewriting your entire application.
- **Second, Prompt Templates:** Rather than hardcoding the same prompt text every time, LangChain lets you define a template with variable placeholders. You supply the values at runtime and the framework assembles the final prompt for you. This makes your prompts reusable, consistent, and easy to maintain across large applications.
- **Third, Memory:** By default, a language model has no memory. Every call is stateless. LangChain adds memory modules that store conversation history and pass it back to the model on each new turn. This is what allows a model to carry context across a full conversation rather than treating every message as if it were the first.
- **And lastly, Chains:** A chain is a sequence of steps connected together. The output of one step becomes the input of the next. This is where LangChain gets its name, and it is what allows you to build multi-step workflows that go well beyond a single prompt and response.

PART 4: Why This Matters in Practice

In 2025 and 2026, LangChain is one of the most widely used frameworks for building production AI applications across industries. Companies are using it to build customer support systems that remember user history, document analysis tools that retrieve and reason over large knowledge bases, and research assistants that chain together multiple models and data sources to produce structured reports.

In each of these cases, the underlying language model is just one piece of the system. LangChain provides the connective tissue that makes the whole thing work reliably at scale.

PART 5: What You Will Build in This Module

In this module, you will move from concept to code. You will connect multiple language models through LangChain and compare their outputs. You will build reusable prompt templates with dynamic variables. You will implement four different memory types and observe how each one changes the behavior of a multi-turn conversation.

The lab will bring all of these components together in a single working application.

COE_MSDAE_DADS5250_M06V01_AI Agents

PART 1: INTRODUCTION

I play chess occasionally. When you are new to the game, you move one piece and then you wait. You react to whatever your opponent does. But experienced players do not think one move ahead. They think five or six moves ahead. They plan contingencies. They adapt when something unexpected happens. That is exactly what separates a chain from an AI agent. A chain executes the moves you planned in advance. An agent looks at the board and decides its own next move.

PART 2: Chains vs. Agents

Remember, a chain is a fixed sequence of steps. You define step one, step two, step three, and the system executes them in that order every time. Chains are reliable, predictable, and efficient for tasks where the process never changes.

But many real-world tasks do not have a fixed process. The steps required depend on what the model discovers along the way. A user might ask a question that requires a database lookup, then a calculation, then a web search, in an order that nobody could fully predict in advance.

This is where agents come in. An agent does not follow a script. It receives a goal, reasons about what needs to happen to achieve that goal, selects the appropriate tool, observes the result, and then decides what to do next. The process is dynamic. Each action informs the next one.

PART 3: What Is a Tool?

So, what is a tool in this case? The capabilities of an agent are defined by the tools it has access to. A tool is a function that the agent can call when it needs to. It could be a calculator for arithmetic, a web search for current information, a database query for structured data, or an API call to an external service.

The agent does not use all tools on every request. It reads the user's input, reasons about which tool is appropriate for the current step, calls that tool, and uses the result to decide what to do next. If the result answers the question, it stops. If more work is needed, it continues.

This is what makes agents fundamentally different from chains. The path through the tools is not predetermined. The agent chooses it in real time based on what it finds.

PART 4: How Multi-Step Reasoning Works

The reasoning process that most LangChain agents use is called ReAct, which stands for Reasoning and Acting. It works as a loop.

First, the agent reasons. It asks itself: what do I know, and what do I need to find out? Second, it acts by calling the most appropriate tool. Third, it observes the result that comes back. Fourth, it reasons again based on the new information and decides whether the task is complete or another action is needed.

This loop continues until the agent has enough information to produce a final answer. You can observe this thought process directly in the console output because each reasoning step is printed as the agent works through the problem.

PART 5: When to Use a Chain and When to Use an Agent

Choosing between a chain and an agent is a design decision, not a default. Chains are the right choice when your workflow is predictable, the steps are always the same, and cost control is critical because each agent reasoning step consumes tokens.

Agents are the right choice when the task requires dynamic decision making, when the steps cannot be predetermined, or when the system needs to handle a variety of different user queries through a single interface.

In practice, many production systems combine both. Fixed, well-understood workflows run as chains. Flexible, open-ended tasks run through agents. Knowing when to use each is a core skill for any AI engineer.

PART 6: Module Preview and Closing

In this module, you will build agents in LangChain, equip them with tools, and observe the ReAct reasoning process in action. The lab will take you from a simple math solver to a market analysis agent that reads stock data, calculates moving averages, and produces investment recommendations.

The chess player does not win by memorizing one sequence of moves. The chess player wins by reasoning through each position as it develops. That is the capability this module builds.

Let us get started.

COE_MSDAE_DADS5250_M08V01_RAG

COE_MSDAE_DADS5250_M10V01_n8n

COE_MSDAE_DADS5250_M11V01_Agentic AI + LangGraph

COE_MSDAE_DADS5250_M13V01_AI Ethics